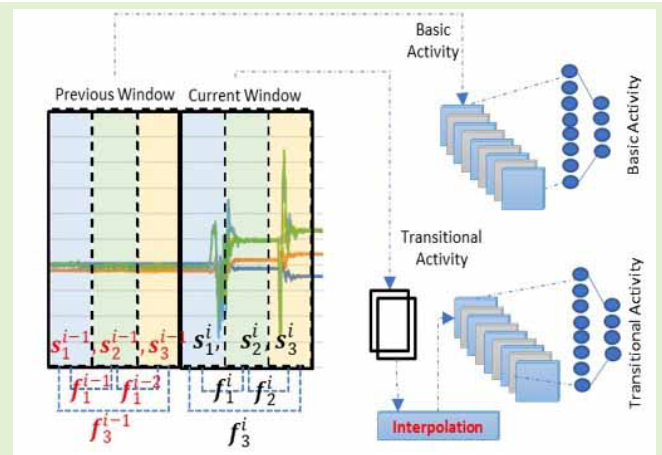


Similarity Segmentation Approach for Sensor-Based Activity Recognition

AbdulRahman M. A. Baraka and Mohd Halim Mohd Noor¹

Abstract—The fixed sliding window is the commonly used technique for signal segmentation in human activity recognition (HAR). However, the fixed sliding window may not produce optimal segmentation because human activities have varying durations, especially for transitional activities (TAs). This is because a large window size may contain activity signals belonging to different activities, and a small window size may split the activity signal into multiple windows. Furthermore, the fixed sliding window does not consider the relationship between adjacent windows, which may affect the performance of the HAR model. In this study, we propose a similarity segmentation approach (SSA) that exploits the temporal structure of the activity signal within the window segmentation process. Specifically, the proposed approach segments each window into subwindows and extracts the inner features by measuring the similarity between them. The inner features are used to measure the dissimilarity between the adjacent windows. The proposed approach is able to distinguish between transitional and nontransitional windows, which achieves more effective segmentation and classification processes. Two public datasets are used for the evaluation. The experimental results show that the proposed approach can distinguish TAs from basic activities (BAs) at 97.65% accuracy, which enhanced the accuracy of TAs recognition compared to the fixed sliding window by 33.41%. Also, our approach achieved accuracy for activity recognition of 92.71% and 86.65% for both datasets, respectively, which exceeds the fixed sliding window by 2.29% and 3.93% for both datasets, respectively. These results are significant and exceed the accuracy of the state-of-the-art models.

Index Terms—Human activity recognition (HAR), signal segmentation, transitional activity (TA).



I. INTRODUCTION

RECENTLY, human activity recognition (HAR) has become one of the most important areas of research and the focus of researchers due to its involvement in many fields, such as the Internet of Things (IoT) applications [1], healthcare [2], security, sport [3], and smart home [4]. HAR is concerned with the ability to recognize human's daily activities to understand the user's behaviors. Human activities can be divided into basic activities (BAs) and transitional activities (TAs)

Manuscript received 30 April 2023; revised 8 July 2023; accepted 12 July 2023. Date of publication 19 July 2023; date of current version 31 August 2023. This work was supported in part by the Ministry of Higher Education Malaysia for Fundamental Research Grant Scheme under Project FRGS/1/2019/ICT02/USM/02/1. The associate editor coordinating the review of this article and approving it for publication was Dr. Varun Bajaj. (Corresponding author: Mohd Halim Mohd Noor.)

AbdulRahman M. A. Baraka was with the Faculty of Technology and Applied Sciences, Al-Quds Open University (QOU), Gaza Strip 970, Palestine. He is now with the School of Computer Sciences, Universiti Sains Malaysia, Pulau Pinang 11800, Malaysia (e-mail: abarakah@qou.edu).

Mohd Halim Mohd Noor is with the School of Computer Sciences, Universiti Sains Malaysia, Pulau Pinang 11800, Malaysia (e-mail: halim-noor@usm.my).

Digital Object Identifier 10.1109/JSEN.2023.3295778

(or postural transitions). The BAs are those performed by a human in daily life and can be classified into two categories: static and dynamic activities. Static activities do not contain motions during their performance, such as walking, jumping, and running. Dynamic activities contain motions during their performance, such as standing and sitting activities. The TAs occur between two BAs, such as stand-to-sit and sit-to-stand. Typically, BAs are performed for a long duration, while TAs have a low incidence and are carried out in a short period of time averaging around 3.7 s [5]. This low duration may lead to difficulty detecting and recognizing TAs [6], [7]. Thus, few researchers considered the TAs in their HAR models. However, TAs are essential in various applications, such as pervasive healthcare and activities monitoring. The importance of detecting and recognizing TAs manifests in the following applications: identifying falls in the elderly [8], separating the different successive activities [9], and segmenting the continuous data signal better, which leads to improving the accuracy of HAR recognition. Furthermore, if the TAs are not considered in the HAR model, it could introduce data fluctuations in the signals, consequently affecting the model's performance.

Recently, inertial sensors, such as accelerometers and gyroscopes, have become popular embedded sensors in wearable devices of various types [10]. These sensors capture the body motions while the person performs an activity. The accelerometer measures the acceleration along the three axes, while the gyroscope measures the rotation around the three axes. These continuous signals must be segmented into a sequence of windows to perform the features extraction task, either manually or automatically, followed by classifying the windows into activities. The fixed-size sliding window is the most commonly used technique for signal segmentation in HAR, which segments the signals into equal-size segments or windows. However, using a fixed window is not effective and optimal due to the varying lengths of human activity signals. A small window size could not capture the general characteristics of the activity signal, and a large window size causes the window to contain data belonging to different activities. Thus, using a fixed window size for signal segmentation is not effective for activity recognition [6], [11].

In this article, we propose a similarity segmentation approach (SSA) for the signal segmentation process. The proposed approach analyzes two adjacent windows to exploit their temporal relationship. To this end, the proposed approach further splits the windows into subwindows and extracts the features by calculating the inner similarity between the subwindows. Then, the dissimilarity between the features of the adjacent windows is measured to detect the boundaries of the window segmentation. Furthermore, we propose two convolutional neural network (CNN) models: one for BAs and the other for TAs. Dividing the recognition task into two models allows each model to focus on learning their own set of activity classes.

The rest of this article is organized as follows. Related work is in Section II. Section III explains our proposed methodology. The experiments and results are discussed in Section IV. Section V is the conclusion.

II. RELATED WORKS

Human activity is person-dependent [12], which means it varies among humans in terms of time and actions. In general, human activity can be categorized into two categories: BA and TAs. The BA is performed daily by humans, whether long (e.g., walking) or short (e.g., jumping). The TA occurs between two BAs. The TA has two main characteristics compared with the BA: low-performing duration time and low incidence (e.g., sit-to-stand). These characteristics led the researchers to neglect the TAs in HAR.

As aforementioned, the fixed sliding window is the most commonly used technique for segmenting the activity signal in activity recognition [13]. Despite these limitations, several studies adopted fixed-size sliding window to segment their sensor data [14], [15], [16]. Also, some studies [3], [17] investigated the effect of different window sizes on the signal segmentation task. Banos et al. [18] analyzed and evaluated the impact of fixed window size on HAR model performances and show that a small window size (0.25–0.5 s) with nonoverlap is better than a large window size for recognition performance.

Numerous studies proposed an adaptive and dynamic window size to overcome the limitation of the fixed-size sliding window method. In the dynamic sliding window, the sensor data are segmented into different sizes of windows based on specific features. Alhammad and Al-Dossari [2] proposed a segmentation method based on their assumption that the *Y*-axis of the acceleration signal can present the boundaries of all activity types. The method depends on choosing the peaks with the minimum value and selecting the highest value of the valley and also finding the minimum distance between peaks by using a threshold value. Then, the signal characteristics are analyzed using several features to determine the activity boundaries. Akbari et al. [19] proposed a hierarchical signal segmentation method by segmenting the data using a large fixed-window size and extracting the features to predict the activity. If the window is suspected to contain multiple activity signals, the method divides the window into smaller subwindows to fine-tune the label assignment for each subwindow. Xiao et al. [20] proposed a deep learning CNN-based algorithm for activity segmentation. The sensor data are segmented into sets of bins using a sliding window method, and then, each bin is assigned a label. The network identifies the start and end of activity by detecting the changes in the labels that appear in the set of bin labels. He et al. [21] adopted a weakly supervised learning setting to avoid expensive annotation effort. The framework uses support measure machines (SMMs) and features extraction, which is based on kernel embedding of distributions. Signal segmentation, feature extraction, and activity classification tasks are performed jointly.

However, all the aforementioned studies do not consider the TAs in their experiments, which could lead to obtaining a low accuracy in recognizing the TAs. Therefore, some studies present the segmentation method considering the postural TAs.

Noor et al. [11] proposed an approach that adjusted the adaptive window size by modeling the windows using multivariate Gaussian distribution. Initially, the segmentation is performed using a fixed window size; then, the window is classified into TA or non-TA. If the window is a TA, the adaptive sliding window is activated and the window size is expanded by calculating the belonging probability of the window to a specific activity. Lone et al. [22] used different methods for features extraction and selection methods and five different types of machine learning classification algorithms to classify the BAs and postural transitions. Li et al. [6] present a signal segmentation method by segmenting the data into multiple fragments. After extracting the features from the fragments, K-means clustering is used to classify the fragments. The adjacent fragments that have the same category are combined. The TAs' fragments are separated from Bas' fragments using the duration period of the fragment. Shi et al. [23] distinguished TA windows from BA windows through the standard deviation trend analysis (STD-TA) method. The sliding window method segments the signals, and the statistical characteristics of segments are extracted and then use the support vector machine (SVM) classifier for the activity classification.

The previous studies utilized features engineering to extract and select the features. For improvement, some studies have

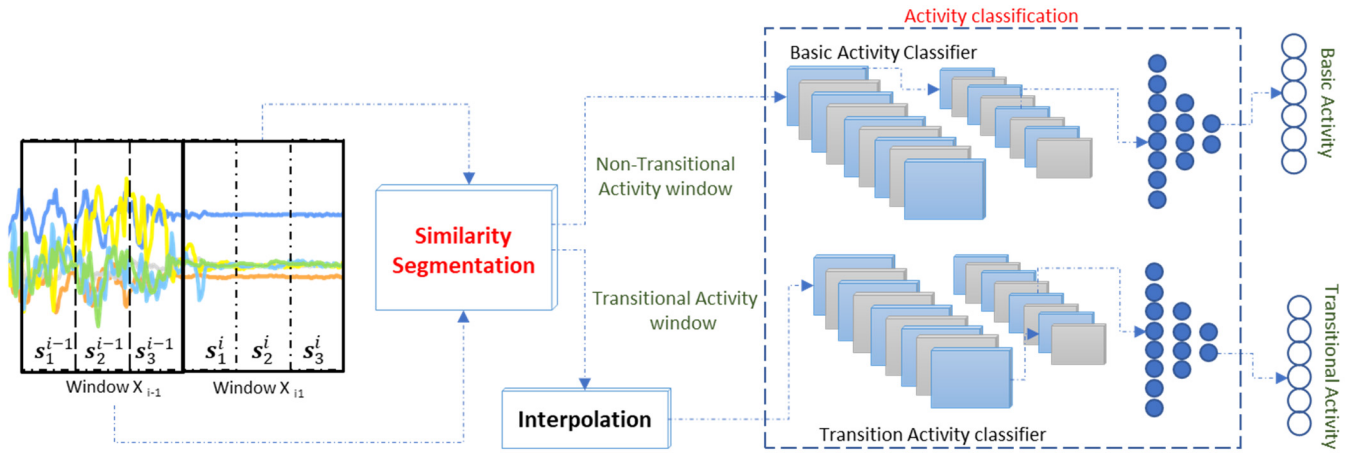


Fig. 1. Overview of our proposed SSA.

adopted deep learning models in order to extract and select the features automatically. Irfan et al. [24] integrated three deep learning models to classify the activity windows. The feature data are passed into the models simultaneously and then fuse the results by using the class probabilities of each model. Noor [25] proposed an unsupervised feature learning method based on a denoising autoencoder, which aims to extract and select the discriminative features for the activity recognition task. The data are segmented using an adaptive sliding window method presented in [11]. Then, the extracted features are represented by denoising the autoencoder using 1-D convolutional and 1-D max-pooling layers.

Furthermore, some studies adopted long short-term memory (LSTM) to model the temporal sequences between windows. Xia et al. [26] built a classification model consisting of two layers of LSTM followed by CNN layers. A fixed sliding window is used for the segmentation task. The model achieved a significant performance; however, the TAs were not considered in the study. Noor et al. [27] proposed a hybrid deep learning model, which uses deep temporal Conv-LSTM architecture to utilize both temporal features and the relationship of windows.

However, most studies treated each window independently regardless of any temporal relationship between the windows. This could lead to a nonoptimal segmentation method. To overcome these issues, we proposed an SSA based on similarity features and considering the temporal relationship between adjacent windows. This allows the proposed approach to distinguish the TA windows from the BA windows, thus enhancing activity recognition performance.

III. PROPOSED METHODOLOGY

The signals from the wearable devices are segmented into a sequence of windows using the fixed sliding window $X = \{x^1, x^2, \dots, x^N\}$, where N is the total number of windows, $x^i \in \mathbb{R}^{w \times C}$, w is the window size, and C is the number of sensor's channels. In this study, the approach of activity classification consists of two stages: the SSA and the activity recognition. The block diagram of the proposed approach is given in Fig. 1.

First, the signals are split into multiple successive windows, and then, the windows are passed into our proposed SSA. The SSA can distinguish between non-TA (basic) and TA windows by extracting two feature types: inner similarity features and adjacent window dissimilarity features. Then, the non-TA windows are fed to the BA classifier, and the TA windows are fed to the TA classifier after applying the interpolation method. The BA classifier classifies the nontransitional windows into BAs, while the TA classifier classifies the TA windows into TAs.

Specifically, the SSA aims to distinguish whether the window belongs to a TA or to a non-TA. To this end, two operations are performed. The first operation is called inner similarity, which extracts the similarity features of the window by measuring the data homogeneity within each window. That is done by dividing the window into three equal-sized subwindows and then measuring the similarity of signals between these subwindows. The second operation is called adjacent window dissimilarity. This operation measures the dissimilarity between the features of the current window and those of the previous window. The current window is the last window read from the sensors and segmented by the fixed sliding window method.

We refer to the current window as x^i . The previous window is the window that precedes the current window, and we refer to it as x^{i-1} . If the degree of dissimilarity between the two adjacent windows is small (i.e., the current and the previous windows), this is an indication that there is no TA in either of both windows, but if the degree of dissimilarity is high, it indicates the presence of TA between one of the two windows. Based on the degree of dissimilarity, we can distinguish between the TA window from the BA window. Also, we can detect the boundaries of the TAs, which are presented by multiple adjacent windows. In order for the size of these adjacent transitional windows to be equal to the fixed window size, we combine them together into one window while maintaining their pattern using the linear interpolation method.

The second stage aims to recognize the activity type in the window through two classifiers: the first classifier aims to classify TAs. The other classifier aims to

classify BAs. Each classifier is implemented using CNN deep learning.

A. Similarity Segmentation Approach

Our proposed SSA aims to segment the activity signals by distinguishing between TA and non-TA (basic) windows using inner similarity features and dissimilarity between two adjacent windows. The proposed algorithm is shown in Algorithm 1.

Initially, the signal is segmented using the sliding window method with a fixed window size w . The window size is the input size of the two classifiers, the BA classifier and the TA classifier. The algorithm accepts two adjacent windows as inputs, i.e., the previous window ($\mathbf{x}^{i-1}$) and the current window ($\mathbf{x}^i$). Next, a flag (is_{ta}) is defined to represent the detection of a TA window. By default, the flag is set to false, as shown in line 1. Then, the normalization process is performed on each window (line 2). Each window is split into three equal-sized subwindows; thus, it is recommended that the window size is divisible by 3. The inner similarity features are extracted by calculating the distances between each subwindow for each window. Then, the dissimilarity between the features of both windows is measured, which results in two values (b) and (e), as shown in line 4. These two values are derived based on the dissimilarity measure between the two adjacent windows, which will be explained in Sections III-B and III-C.

The (b) value is used to detect the beginning of the TA by comparing it with the beginning threshold (b_{ts}). In the case where is_{ta} is false, if the (b) value is greater than the threshold (b_{ts}), the previous window is considered as BA window, and the current window is considered the beginning of a TA window, whereby the current window is added to a buffer $\mathbf{x}_{ta}$ (lines 7 and 8). Otherwise, the previous window is considered a BA window (line 10).

In the case where is_{ta} is true (a transitional window is detected), the algorithm starts aggregating the transitional windows by adding the current window to the buffer $\mathbf{x}_{ta}$ (line 12). The process of aggregation is stop or continue based on (e). Specifically, the algorithm compares (e) with the ending threshold (e_{ts}). As shown in line 13, if the value (e) is higher than the threshold (e_{ts}), we reach the end of the TA window and aggregate the previous window as the ending of the TA. All the detected transitional windows are aggregated in $\mathbf{x}_{ta}$ representing a single transitional window. Then, the algorithm performs interpolation on the aggregated window to resize it to the fixed window size to match the input size of the classifier (line 15). Otherwise, the window aggregation will continue in the next round.

In some cases, the process of aggregating the windows exceeds the maximum length of the TA. This occurs when the algorithm fails to detect the end of the TA window. In this case, the buffer $\mathbf{x}_{ta}$ is assumed to contain both TA and BA signals. To resolve this, the algorithm assumes the first window in the buffer to be the TA window, and the remaining windows are considered as BA windows (lines 22–26). In some cases, we found that two adjacent windows have (b) values higher than the threshold (b_{ts}). This case occurs when both windows

Algorithm 1 Similarity Segmentation Approach

Input: previous window $\mathbf{x}^{i-1}$, current window $\mathbf{x}^i$.

Output: distinguish between transitional activity window from basic activity window.

i : the index of window.

c : number of adjacent transitional windows.

b_{ts} : threshold of the beginning of transitional activity window (b_i).

e_{ts} : threshold of the ending of transitional activity window (e_i).

is_{ta} : flag to determine the transitional activity is detected or not.

max_{TAlen} : maximum number of transitional windows.

$\mathbf{x}_{ta}$: the detected transitional activity windows.

norm(x): perform the normalizing data for x .

interpolate(x): perform the interpolation method on x .

```

1:   $is_{ta} = \text{False}$ 
2:   $norm(\mathbf{x}^i), norm(\mathbf{x}^{i-1})$ 
3:  If window  $\mathbf{x}^i$  is available Then
4:     $b^{i-1}, e^i = \text{compute\_dissimilarity}(\mathbf{x}^{i-1}, \mathbf{x}^i)$ 
5:    If  $is_{ta} = \text{False}$  Then
6:      If ( $b^{i-1} > b_{ts}$ ) Then
7:         $is_{ta} = \text{True}$ 
8:        add  $\mathbf{x}^i$  to  $\mathbf{x}_{ta}$ 
9:      End If
10:     send  $\mathbf{x}^{i-1}$  to the basic activity classifier.
11:    Else:
12:      add  $\mathbf{x}^i$  to  $\mathbf{x}_{ta}$ .
13:       $c = c + 1$ 
14:      If ( $e^i > e_{ts}$ ) Then
15:        interpolate  $\mathbf{x}_{ta}$ 
16:        send  $\mathbf{x}_{ta}$  to the transitional activity classifier.
17:        clear  $\mathbf{x}_{ta}$ 
18:         $is_{ta} = \text{False}$ 
19:        reset counter  $c = 0$ 
20:      End If
21:      If  $c > max_{TAlen}$  Then
22:        send  $\mathbf{x}_{ta}^0$  to the transitional activity classifier.
23:        send  $\mathbf{x}_{ta}^1$  to  $\mathbf{x}_{ta}^c$  to the basic activity classifier.
24:        clear  $\mathbf{x}_{ta}$ 
25:         $is_{ta} = \text{False}$ 
26:        reset counter  $c = 0$ 
27:      End If
28:    End If
29:  End If

```

have mixed samples of TA and non-TAs. However, in this case, we consider the latest window to be the beginning of the TA window.

B. Inner Similarity (Intrawindow Similarity)

Although TAs are short and infrequent, they have distinct patterns that are different from BA patterns, as shown in Fig. 2.

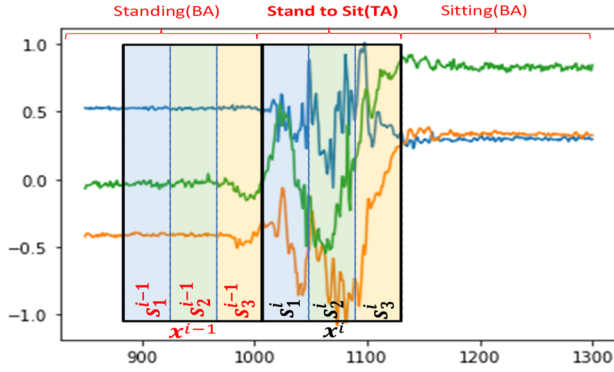


Fig. 2. Differential pattern for the transition activity and BA signals, where window (x^i) is a transition activity window and (x^{i-1}) is a BA window with their subwindows.

The TA pattern is different from the beginning to the end of the window. Therefore, we propose a set of features to represent the inner characteristics of the window based on similarity measurements. The window features are extracted by measuring the similarity within the window and called the inner similarity. To this end, each window is further split into three subwindows (s_1 , s_2 , and s_3), as shown in Fig. 2. The inner similarity is calculated between the subwindows using a distance function. Given a window x^i , the inner similarity measurements of the window $f^i = (f_1, f_2, f_3)$ are given as follows:

$$f_1 = d(s_1, s_2) \quad (1)$$

$$f_2 = d(s_2, s_3) \quad (2)$$

$$f_3 = d(s_1, s_3) \quad (3)$$

where $d(s_k, s_l)$ is the distance function to compute the inner similarity. Theoretically, any common distance function, such as Euclidean distance, Manhattan (City Block) distance, and Cosine similarity. In this study, the Manhattan distance is used to measure the inner similarity. Manhattan distance calculates the differences between two coordinates of paired objects [28]. Manhattan distance has been shown to outperform the Euclidean distance methods in terms of performance index and classification [28]. The Manhattan distance is calculated as the sum of the absolute differences between the two vectors, which is defined as follows:

$$d(s_k, s_l) = \sum |s_k - s_l| \quad (4)$$

where d refers to the distance between two subwindows (s_k, s_l). Since Manhattan distance deals with two vectors, the subwindows are reshaped from $(w/3) \times C$ size into a vector with size $((w/3) \times C) \times 1$ before calculating the Manhattan distance, where C is the number of sensor's channel.

The value of the inner similarity feature is affected by the rate of motion within the window. The more motion within the segment, the higher difference between the subwindows, which increases the value of this feature. Conversely, the less motion within the window, the smaller difference between the subwindows and decreased the feature value. Since the dynamic activities have more motions compared to the static activities, the feature values of the dynamic activities are higher than the feature values of the static activities.

C. Adjacent Windows Dissimilarity

The signal characteristics differ from TA to BAs (whether the activities are static or dynamic). While the signal of static activities is usually low frequency with low amplitude, the signal of dynamic activities is composed of high frequency with high amplitude. On the other hand, the signal of TA is high frequency, but relatively much shorter compared to the BA. Therefore, TA features differ from BA features. A high difference between the features of two successive windows indicates the presence of a TA window, either at the beginning or at the end of the window. Thus, the proposed segmentation approach exploits these temporal characteristics by deriving two measures from the inner similarity features. This step aims to calculate the degree of dissimilarity between the inner similarity features of two consecutive windows.

To this end, two dissimilarity values are calculated between two adjacent windows. The first value is b , which aims to determine whether the current window is the beginning of the TA or not. This value is measured by calculating the absolute difference between the minimum value of previous window features and the maximum value of current window features. This value is high if the current window is the beginning of TA and the previous window is the ending of BA. The second value e aims to determine whether the current window is the end of a TA or not. This value is measured by calculating the absolute difference between the minimum value of the current window features with the maximum value of the previous window features. The ratio of this difference is calculated based on the lowest value of features for the current window. Given an inner similarity feature vector f^i and f^{i-1} for the current and previous windows, respectively, the adjacent windows dissimilarity is calculated as follows:

$$b^{i-1} = \frac{|\min(f_1^{i-1}, f_2^{i-1}, f_3^{i-1}) - \max(f_1^i, f_2^i, f_3^i)|}{\min(f_1^{i-1}, f_2^{i-1}, f_3^{i-1})} \times 100 \quad (5)$$

$$e^i = \frac{|\min(f_1^i, f_2^i, f_3^i) - \max(f_1^{i-1}, f_2^{i-1}, f_3^{i-1})|}{\min(f_1^i, f_2^i, f_3^i)} \times 100 \quad (6)$$

where b^{i-1} is calculated based on the measure of the difference between the maximum feature value of the current window and the minimum feature value of the previous window, relative to the minimum feature value of the previous window. Note that the feature values f^i are based on the similarity between the subwindows (inner similarity). Thus, b^{i-1} will be large if the denominator is low and the numerator is high.

e^i is calculated based on the measure of the difference between the maximum feature value of the previous window and the minimum feature value of the current window, relative to the minimum feature value of the current window. Thus, e^i will be large if the value of the denominator is low and the numerator is high.

Therefore, the difference between b and e is that the denominator for the b value is based on the features of

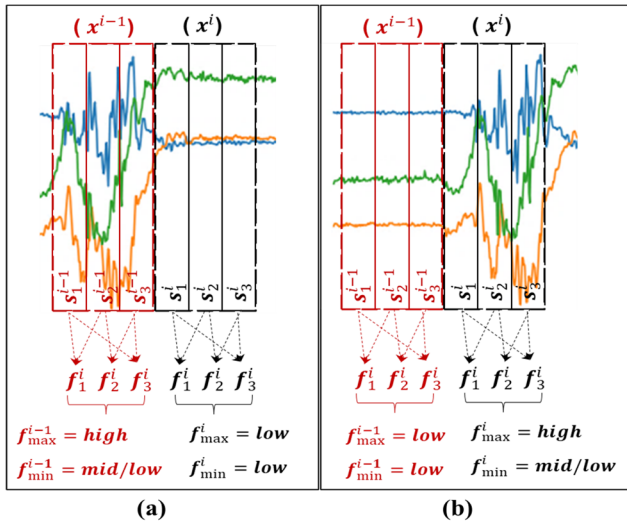


Fig. 3. Differentiation of the value of the maximum and minimum of the features between two cases of two successive windows. (a) Basic activity preceded by a transitional activity result in low feature values in the current window and mid or high feature values in the previous window. (b) Transitional activity preceded by a basic activity result in mid or high feature values in the current window and low feature values in the previous window.

the previous window, while the denominator for the e value is based on the features of the current window. Also, the maximum and minimum feature values in the numerator of both b and e are swapped between the current and previous windows.

For more clarification, Fig. 3 illustrates two cases that show the differences in calculation between b^{i-1} and e^i values.

The first case is shown in Fig. 3(a). The current window (x^i) is a BA, while the previous window (x^{i-1}) is a TA. Thus, the maximum and minimum feature values of the current window indicated by $f_{\max}^i$ and $f_{\min}^i$ are low because the distance between subwindows is small since the pattern of signal data is similar. For the previous window, the maximum feature value $f_{\max}^{i-1}$ is high and the minimum feature value $f_{\min}^{i-1}$ is low or medium because the distance between subwindows is large since the pattern of signal data is different. Based on (5) and (6), the minimum and maximum feature values are used to determine b^{i-1} and e^i . Therefore, in this case, b^{i-1} and e^i values will be small and large, respectively.

In the second case, as shown in Fig. 3(b), the current window (x^i) is a TA and the previous window (x^{i-1}) is a BA. Thus, $f_{\max}^i$ value is high and $f_{\min}^i$ is low, while both $f_{\max}^{i-1}$ and $f_{\min}^{i-1}$ values are low. Thus, b^{i-1} and e^i values will be large and small, respectively.

In all cases, the large value of b is used to determine the beginning of the TA, while the large value of e is used to determine the ending of the TA.

Note that the measures of “high, mid, and low” that are used here are to describe the behavior of both values b and e only. The mathematical calculations will have a slightly different effect, especially when calculating the ratio.

All previous steps are listed in Algorithm 2.

Algorithm 2 Compute the Inner Similarity Features and Windows Dissimilarity for Two Adjacent Windows

Input: previous window x^{i-1} , current window x^i

Output: b^{i-1}, e^i

b^{i-1} : is a value that refers to the end of a basic activity and starts a transitional activity.

e^i : is a value that refers to the end of a transitional activity and starts a basic activity.

round() is rounding to the nearest integer.

split(x, c) split window x into c equal-sized sub-windows.

s_1^i, s_2^i, s_3^i : sub-windows of i th window.

1: $s_1^{i-1}, s_2^{i-1}, s_3^{i-1} = \text{split}(x^{i-1}, 3, \text{dim} = 0)$

2: $s_1^i, s_2^i, s_3^i = \text{split}(x^i, 3, \text{dim} = 0)$

3: $f_1^{i-1}, f_2^{i-1}, f_3^{i-1} = \text{calculate distances between subwindows of } x^{i-1}(s_1^{i-1}, s_2^{i-1}, s_3^{i-1})$

4: $f_1^i, f_2^i, f_3^i = \text{calculate distances between subwindows of } x^i(s_1^i, s_2^i, s_3^i)$

5: round all features, $\text{round}(f_1^{i-1}, f_2^{i-1}, f_3^{i-1}, f_1^i, f_2^i, f_3^i)$

6: $b^{i-1} = \frac{\min(f_1^{i-1}, f_2^{i-1}, f_3^{i-1}) - \max(f_1^i, f_2^i, f_3^i)}{\min(f_1^{i-1}, f_2^{i-1}, f_3^{i-1})} \times 100$

7: $e^i = \frac{\min(f_1^i, f_2^i, f_3^i) - \max(f_1^{i-1}, f_2^{i-1}, f_3^{i-1})}{\min(f_1^i, f_2^i, f_3^i)} \times 100$

8: return b^{i-1}, e^i

D. Activity Classifiers

The CNNs have been shown to be effective in many fields and have been widely used in HAR studies [29]. The convolutional layers allow the network to automatically extract features from each window for recognition tasks. Therefore, two CNNs are built with the same architecture to classify the windows. The architecture of the networks is illustrated in Table I. This architecture is similar to the model architectures reported in [13] and [30]. The first network is used to recognize the BA windows, and the second network is used to recognize the TAs. After the SSA distinguishes the window into BA window or TA window, the BA window is passed to the BA classifier, and the TA window is passed to the TA classifier for activity recognition.

IV. EXPERIMENTS AND RESULTS

In this section, we evaluate the performance of the SSA. Also, the impact of similarity segmentation on recognition performance is evaluated for both TA and human activities. All experiments are performed using two different public datasets.

A. Datasets Description

In this study, we used two public benchmark datasets: SBHARPT [7] and FORTH-TRACE [31], to perform our experiments and evaluate the proposed approach.

The SBHARPT dataset is a collection of continuous data of triaxial signals read from the accelerometer and gyroscope sensors, and it is an updated version of the UCI HAR dataset [32]. The dataset contains activity data from 30 subjects who performed 12 activities detailed as follows: six BAs, where

TABLE I
STRUCTURE OF BA AND TA CNN CLASSIFIERS

	Layer	Parameter	Output size
1	Input	Shape size = $6 \times L$	
2	Conv (ReLU)	input=6, output=512, kernel size=3, stride=1	$512 \times w-2$
3	Max Pooling	kernel size=2, stride=2, padding=0, dilation=1	$512 \times (w-2)/2$
4	Dropout	p=0.5, inplace=False	$512 \times (w-2)/2$
5	Conv (ReLU)	Input=512, output=1024, kernel size=3, stride=1	$1024 \times (w-4)/2$
6	Average Pooling	Kernel size=2, stride=2, padding=0, dilation=1	$1024 \times (w-4)/4$
7	Dropout	p=0.5, inplace=False	$1024 \times (w-4)/4$
8	Flatten	start_dim=1, end_dim=-1	$1024 \times (w-4)/4$
9	Fully Connected	input size =21504, output size=150, bias=True	150
10	Fully Connected	input size =150, output size =50, bias=True	50
11	Fully Connected	input size =50, output size =6, bias=True	6

* w IS LENGTH OF WINDOWS.

three static activities (i.e., standing, sitting, and lying down) and three dynamic activities (i.e., walking, walking downstairs, and walking upstairs), and six postural transitions that are occurred between the static postures (i.e., stand-to-sit, sit-to-stand, sit-to-lie, lie-to-sit, stand-to-lie, and lie-to-stand). The signals are captured at a constant sampling rate of 50 Hz by the embedded accelerometer and gyroscope of the smartphone, which was attached to the waist during the execution of the activities. The dataset is split by subjects into 23 subjects for the training stage and seven subjects for the testing stage.

Fig. 4 shows the distribution of the proportion of the number of samples for each activity type in the dataset. Also, it demonstrates that the number of samples for TAs is too low compared to the number of samples for BAs.

The FORTH-TRACE dataset is collected from 15 subjects wearing five Shimmer sensor nodes in five different locations. The sensor node is integrated with a triaxial inertial sensor (accelerometer, gyroscope, and magnetometer), which collected the activity data at a constant sampling rate of 51.2 Hz. The subjects performed 16 activities: seven BAs and nine postural TAs. The raw data of the dataset are split by subjects into 11 subjects for the training stage and four subjects for the testing stage.

In addition, the FORTH-TRACE dataset includes activities with talking, such as sitting-while-taking. The annotation of the dataset differentiates between sitting activity and sitting-while-taking. However, in this study, we consider both activities to be the same type of activity because talking is not captured by the accelerometer and gyroscope, which are mainly used for capturing human motions [33]. Furthermore, some of the postural TAs in FORTH-TRACE occurred

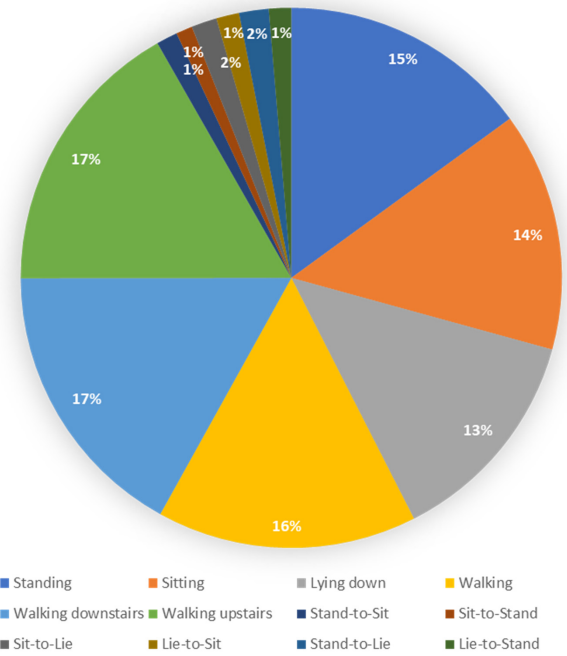


Fig. 4. Number of samples for activity types in the SBHARPT dataset.

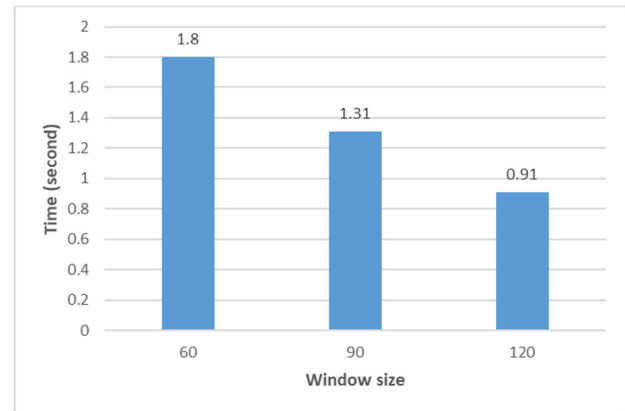


Fig. 5. Running time of our proposed segmentation approach based on window size.

between the dynamic postures, which may lead to the misclassification of TA. To avoid this, we considered the first activity only regardless of the second activity (e.g., climb stairs to walk activity become climb stairs activity only).

B. Experimental Setup

First, the sensor data are preprocessed to remove the missing values. The main variables used from the datasets are the three axes for the accelerometer and the three axes for the gyroscope. The subject id is used for splitting the dataset. The other variables are neglected. All dataset files are combined into one file to present the continuous data signal. Then, the normalization method is performed to standardize the data signals. Then, the data are segmented using the fixed-size sliding window with nonoverlapping because our proposed approach uses windowwise treating. Since the TA's duration is less than the BA's duration, we consider window sizes that

are less than the minimum TA's duration. While the minimum TA's duration time is less than 2.6 s approximately, the signals are segmented by three different window sizes of less than (2.6 s). In this study, three different window sizes are used to evaluate the impact of window size on the performance of the proposed approach, i.e., 60 samples (1.2 s), 90 samples (1.8 s), and 120 samples (2.4 s). A window's size refers to the size of a single window.

The dataset is divided into training and testing datasets. The training set is further divided into the BA and TA training set. The BA training set contains just BA windows, while the TA training set contains just TA windows. The TA windows are aggregated and resized to the size of the window using linear interpolation.

The BA classifier is trained using the BA windows, and the TA classifier is trained using the TA windows. Adam optimizer with a default learning rate (0.001) is used and sets the hyperparameters empirically. The batch size is 64 windows, and the maximum epoch is 180 epochs for the TA classifier and 200 epochs for the BA classifier. The models are trained using cross entropy loss.

To perform the experiments optimally, we select the optimal value of the beginning of TA (b_{ts}) and (e_{ts}) thresholds. The value is selected by some experiments that discuss in Section IV-C. However, the threshold value is changed depending on some factors, such as dataset type and window size. While the maximum average duration of TAs is 4.95 s [7], we set the maximum length of TAs as (10.2 s).

We implemented the proposed method by using Python and the models by using PyTorch on a workstation with CPU Intel Core i5, 16-GB RAM, and GPU GTX1660.

C. Results and Discussion

Here, we will explain and discuss the results of the evaluation experiments. Two types of experiments are performed: the first experiment aims to evaluate the performance of the proposed segmentation approach. The second experiment aims to evaluate the performance of the HAR model, and how our proposed SSA has improved the performance of HAR models compared to the fixed sliding window segmentation method.

The performance of the HAR model is evaluated through the overall accuracy (ac) metric, which is measured as follows:

$$ac = \frac{(TP + TN)}{(TP + FP + TN + FN)} \quad (7)$$

where TP is the true positive, TN is the true negative, FP is the false positive, and FN is the false negative.

The activities performance is evaluated through the metrics: precision (P_c), recall (R_c), and f -score (F_c), which are calculated by the following equations:

$$P_c = \frac{TP}{TP + FP} \quad (8)$$

$$R_c = \frac{TP}{TP + FN} \quad (9)$$

$$F_c = 2 \times \frac{P_c \times R_c}{(P_c + R_c)} \quad (10)$$

where c is the activity class.

TABLE II

ACCURACY RESULTS FOR THE DETECTION EXPERIMENTS OF THE BEST VALUE OF BEGIN THRESHOLD b_{ts} FOR EACH WINDOW SIZE. BEST RESULT IS IN BOLD

RESULT IS IN BOLD.						
SBHARPT						
Window size	Threshold					
	600	700	780	800	900	1000
60	97.04%	97.18%	97.17%	97.23%	96.89%	97.01%
90	97.34%	97.53%	97.65%	97.63%	97.51%	97.37%
120	97.26%	97.35%	97.43%	97.37%	97.13%	97.01%
FORTH-TRACE						
Window size	Threshold					
	800	1000	1200	1400	1600	1800
60	97.49%	97.87%	98.01%	98.07%	98.12%	98.00%
90	97.66%	97.98%	98.12%	98.23%	98.22%	98.18%
120	97.81%	98.04%	98.12%	98.21%	98.12%	98.13%

1) *Performance of Non-TA and TA Window Detection*: Three groups of experiments are performed in this section. The first group aims to find the optimal values for the thresholds. The second group aims to find best similarity distance method for the proposed approach. The third group aims to evaluate the performance of SSA.

Initially, we performed the first group of experiments to find the optimal threshold value of the beginning (b_{ts}) and ending (e_{ts}) of TA windows. The experiments are performed using three different window sizes, 60, 90, and 120 samples. Also, the experiments are performed using different values of thresholds (b_{ts}), i.e., the threshold value starts at 600 and increases by 100 for the SBHARPT dataset and starts at 800 and increases by 200 for the FORTH-TRACE dataset. Table II shows the results of recognition accuracies for different window sizes and different threshold values. As shown in the table, the best threshold value of beginning TA (b_{ts}) is 780 and 1400 for SBHARPT and FORTH-TRACE datasets, respectively.

The second experiments group is performed to determine the optimal similarity distance method that obtains the higher result with our proposed approach. Let s_1 and s_2 are two subwindows for window i . The most common similarity distance metrics are considered in other studies [34], [35], which includes City Block (Manhattan) and

$$\text{Euclidean} \quad \sqrt{\sum (s_1 - s_2)^2} \quad (11)$$

$$\text{Chebyshev} \quad \max |s_1 - s_2| \quad (12)$$

to analyze the similarity between subwindows features.

It is worth noting that distance function such as Hamming [36] or Jaccard [37] functions can be used for categorical or discrete data. The experiments are performed using 90 samples window size since it achieved the best accuracy for the SBHARPT dataset. The results show that using Manhattan (City block) method in the proposed approach achieved the best accuracy compared with other similarity distance methods, as shown in Table III.

The third group of experiments is performed to evaluate SSA performance in terms of distinguishing the TA and

TABLE III

ACCURACY RESULTS OF DISTINGUISHING BETWEEN TA AND NON-TAS USING VARIOUS SIMILARITY METHODS FOR THE SBHARPT DATASET WITH 90 SAMPLES WINDOW SIZE. BEST RESULT IS IN BOLD

Distance Similarity Method	Accuracy
Euclidean	90.71%
Manhattan	97.65%
Chebyshev	94.58%

TABLE IV

ACCURACY RESULTS OF PROPOSED SSA TO DISTINGUISH BETWEEN TA AND NON-TAS FOR BOTH DATASETS WITH THREE DIFFERENT WINDOW SIZES. BEST RESULT IS IN BOLD

	Window size		
Dataset	60	90	120
SBHARPT	97.17%	97.65%	97.43%
FORTH-TRACE	98.07%	98.23%	98.21%

TABLE V

PRECISION, RECALL, AND *f*-SCORE RESULTS OF THE PROPOSED SSA TO DISTINGUISH BETWEEN TA AND NON-TAS FOR BOTH DATASETS WITH THREE DIFFERENT WINDOW SIZES

	SBHARPT			FORTH-TRACE		
	Window size=60					
Activity	Precision	Recall	F- Score	Precision	Recall	F- Score
Basic	99.0%	98.0%	98.0%	99.0%	99.0%	99.0%
Transition	81.0%	85.0%	83.0%	65.0%	75.0%	70.0%
	Window size=90					
Basic	99.0%	98.0%	99.0%	99.0%	99.0%	99.0%
Transition	83.0%	90.0%	86.0%	66.0%	83.0%	74.0%
	Window size=120					
Basic	99.0%	98.0%	99.0%	99.0%	99.0%	99.0%
Transition	80.0%	91.0%	85.0%	66.0%	82.0%	73.0%

non-TA windows (binary classification). To achieve this, the dataset is annotated as follows. The BA windows are annotated with 0 and the TA windows are annotated with 1. We performed three experiments using three different window sizes, 60, 90, and 120 samples to evaluate the impact of the window size on the performance of the proposed segmentation approach.

The performance of the segmentation approach is evaluated in terms of accuracy, precision, recall, and *f*-score metrics. Based on the experiments, it was found that the best window size is 90, with an accuracy of 97.65% and 98.23% for the SBHART and FORTH-TRACE datasets, respectively. The recognition accuracy for each window size is shown in Table IV.

Table V illustrates the recall, precision, and *f*-score measures in detecting non-TA and TA classes for the SBHARPT and FORTH-TRACE datasets, respectively. From the results, the best window size is 90, which got the best results for overall evaluation metrics for both datasets. Also, the results of the metrics of BAs are higher than the TAs. This is due to the fact that the number of instances of TAs is lower than the number of BAs instances. Also, the appearance of TA is less than that of BAs.

TABLE VI

COMPARISON BETWEEN THE ACCURACY (%) OF ACTIVITY RECOGNITION BETWEEN THE FIXED SLIDING WINDOW AND SSA METHODS WITH DIFFERENT WINDOW SIZES FOR BOTH DATASETS. BEST RESULT IS IN BOLD

	Window size					
	60		90		120	
Dataset	Sliding window	SSA	Sliding window	SSA	Sliding window	SSA
SBHARPT	89.75	91.91	90.42	92.71	91.2	91.7
FORTH-TRACE	80.83	82.85	82.06	84.75	82.72	86.65

In addition, the complexity of our proposed segmentation approach is $O(n^2)$ and the space of input is two windows with size ($w \times c$), where w is the window size and c is the channel numbers. Fig. 5 shows the comparison of running time for our proposed segmentation approach based on window size. The total number of windows that proceed are 13 593, 9062, and 6796 windows for window size 60, 90, and 120 samples, respectively.

In summary, the results show that the proposed SSA is able to effectively distinguish TA windows from BA windows for both datasets.

2) Activity Recognition: In the third experiment, we evaluate and compare the proposed SSA and the fixed sliding window method in the context of activity recognition. The experiments are performed using three different window sizes 60, 90, and 120 samples to evaluate the impact of window size on the performance of activity recognition. The results of activity recognition accuracy for SSA and fixed sliding window on both datasets are shown in Table VI.

For the SBHARPT dataset, it is observed that the SSA achieved better recognition accuracies for all window sizes compared to the fixed sliding window, whereby the accuracy is improved by 2.16%, 2.29%, and 0.50% for 60 samples, 90 samples, and 120 samples, respectively. The fixed sliding window achieved an accuracy of 89.75%, 90.42%, and 91.2% for window sizes 60, 90, and 120 samples, respectively, while the proposed SSA achieved an accuracy of 91.91%, 92.71%, and 91.70% for window sizes 60, 90, and 120 samples, respectively. Also, the results show that a window size of 90 samples achieved the best recognition accuracy, which is comparable to an existing study reported in [12], and the study found that window sizes ranging from 2 to 2.5 s demonstrate the most accurate detection performance for common sports activities. Therefore, the following comparative analysis and discussion are based on 90 samples window size.

Table VII shows the precision, recall, and *f*-score measures of each activity for both the SSA and the fixed sliding window for the SBHARPT dataset. As shown in Table VII, both the proposed SSA and fixed sliding window achieved a good performance in classifying the BAs, with an *f*-score measure above 95% for all activity classes except sitting and standing. This is because both the sitting and standing activities are static and have close features. Overall, the average *f*-scores of the BAs for fixed sliding window and SSA are 92.83% and 93.67%, respectively.

TABLE VII
ACTIVITY RECOGNITION PERFORMANCE FOR THE SBHARPT
DATASET FOR A WINDOW SIZE 90 SAMPLES USING
SSA AND FIXED SLIDING WINDOW

Activity	Fixed Sliding Window			SSA		
	Precision (%)	Recall (%)	F-Score (%)	Precision (%)	Recall (%)	F-Score (%)
A1	96%	99%	97%	94%	99%	96%
A2	99%	92%	96%	98%	97%	98%
A3	93%	97%	95%	97%	95%	96%
A4	81%	91%	86%	84%	91%	87%
A5	90%	80%	85%	91%	82%	86%
A6	98	99%	98%	98%	100%	99%
BA Average	92.83%	93.0%	92.83%	93.67%	94.0%	93.67%
T1	69%	55%	61%	82%	82%	82%
T2	69%	78%	73%	81%	91%	86%
T3	68%	55%	61%	88%	95%	91%
T4	68%	81%	74%	85%	71%	77%
T5	49%	58%	53%	75%	75%	75%
T6	71%	63%	67%	87%	77%	82%
TA Average	65.67%	65.0%	64.83%	83.0%	81.83%	82.17%

*A1-A6 are basic activity, and T1-T6 are transitional activity.

For TAs, the fixed sliding window performed poorly in classifying the activities with the f -score measures in the range of 61%–74%. This shows that the fixed window size cannot handle the TAs due to their varying lengths. Conversely, the proposed SSA achieved a significantly better performance in classifying the TAs, with f -score measures in the range of 75%–91%. The proposed SSA successfully detects the TA windows and aggregates them to produce more effective segmentation. Overall, the average f -scores of the TAs for fixed sliding window and SSA are 64.83% and 82.17%, respectively. Fig. 6 presents the confusion matrix for activity recognition for window size 90 using the SSA approach.

For the FORTH-TRACE dataset, it is observed that the proposed SSA achieved better recognition accuracies for all window sizes compared to the fixed sliding window, whereby the accuracy is improved by 2.02%, 2.69%, and 3.93% for 60 samples, 90 samples, and 120 samples, respectively. The fixed sliding window achieved an accuracy of 80.83%, 82.06%, and 82.72% for window sizes 60, 90, and 120 samples, respectively. SSA achieved an accuracy of 82.85%, 84.75%, and 86.65% for the window size of 60, 90, and 120 samples, respectively. Also, the results show that a window size of 120 samples achieved the best recognition accuracy. Therefore, the following comparative analysis and discussion are based on the window size of 120 samples. Table VIII shows the precision, recall, and f -score measures of each activity for both SSA and fixed sliding window in the FORTH-TRACE dataset. As shown in Table VIII, both the fixed sliding window achieved a moderate and close performance in classifying the BAs except sitting and standing

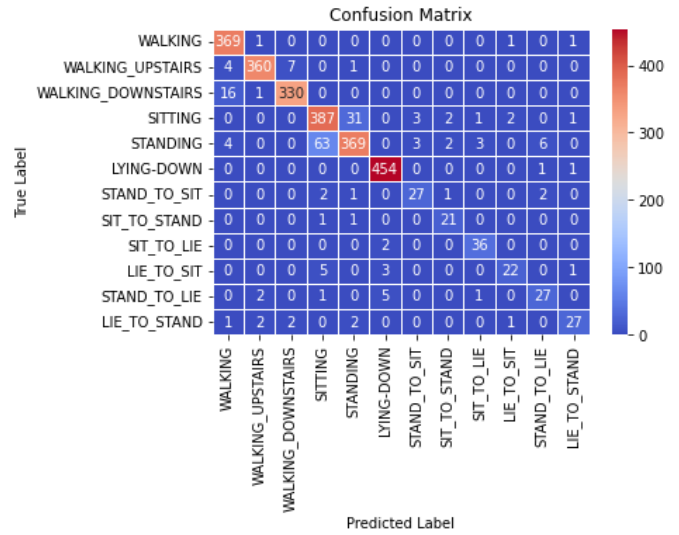


Fig. 6. Confusion matrix of activity recognition results using SSA with window size 90 samples for the SBHARPT dataset.

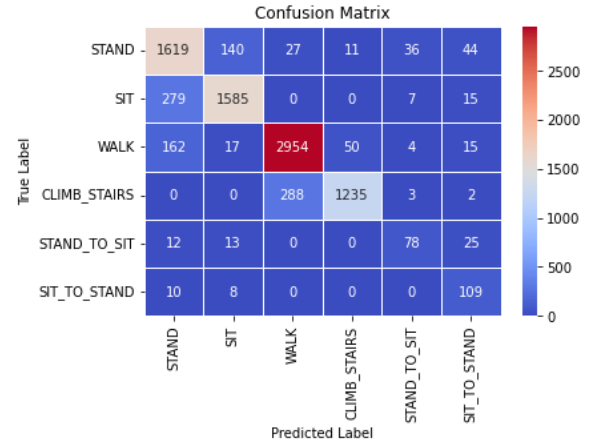


Fig. 7. Confusion matrix for activity recognition results using SSA with window size 120 for the FORTH-TRACE dataset.

activities. The same as SBHARPT dataset, the sitting and standing activities have lower f -score compared to other activity classes. Nevertheless, the proposed SSA achieved better f -score measure for all BA classes compared to the fixed sliding window. Overall, the average f -scores of the BAs for fixed sliding window and SSA are 83.25% and 86.75%, respectively.

For the TAs, the fixed sliding window performed poorly in classifying the TAs, with an f -score measuring 33% and 24% for both TAs, respectively. This shows that the fixed window size cannot handle the TAs due to their varying lengths. Furthermore, the proposed SSA achieved better performance for classifying the TAs, with an f -score measuring 61% and 65% for both TAs, respectively. This low performance is due to combining the talking activity with its corresponding transition activity. Overall, the average f -scores of the TAs for fixed sliding window and SSA are 28.5% and 63%, respectively. Fig. 7 presents the confusion matrix of activity recognition using the SSA approach for the FORTH-TRACE dataset.

TABLE VIII
PRECISION, RECALL, AND f -SCORE RESULTS OF ACTIVITY
RECOGNITION FOR THE FORTH-TRACE DATASET IN
WINDOW SIZE 120 SAMPLES USING SSM
APPROACH AND FIXED SLIDING WINDOW

Activity	Fixed Sliding Window			SSA		
	Precision (%)	Recall (%)	F-Score (%)	Precision (%)	Recall (%)	F-Score (%)
Stand	71	80	75	78	86	82
Sit	87	79	83	90	84	87
Walk	85	93	89	90	92	91
Climb Stairs	93	80	86	95	81	87
Average Basic	84.0	83.0	83.25	88.25	85.75	86.75
Stand-To-Sit	65	22	33	61	61	61
Sit-To-Stand	37	17	24	52	86	65
Average transition	51	19.5	28.5	56.5	73.5	63

3) Comparison With the State-of-the-Art Approaches: One of the main problems facing HAR research is the lack of a benchmark dataset and the lack of standardized evaluation of the HAR model's performance, which leads to an unfair comparison between the state-of-the-art techniques [21]. This is because building a benchmark dataset requires a large number of attributes that differ from one database to another and cannot be standardized, such as sensor types, sensor positions, device types, and activity types. The performance of our model will be compared with the performance of some state-of-the-art models that use the same database.

The SBHARPT dataset is mostly used in studies that consider TA in their activity recognition. Table IX shows the accuracy of the state-of-the-art HAR models on the SBHARPT dataset and the segmentation method for each study. Although some of the existing works show a higher recognition accuracy, the comparison is not completely fair because each study has its own setting or some limitations.

In [22], the BA data are separated from the TA data, and then, each data are classified separately. The fixed sliding window is used for the segmentation process. Furthermore, different machine learning methods are built to obtain the best accuracy for each set. In [5], the TAs are treated as a single class. Thus, the recognition accuracies are actually not representing the model's ability to classify the human activity classes individually.

Irfan et al. [24] and Yulita and Saori [38] used the pre-segmented data of the dataset, which consists of a set of statistical features for each activity instead of the raw sensor data as in our study. Furthermore, the presegmented data have a much smaller number of windows. Thus, it is not appropriate to compare their experimental results with ours. Noor et al. [27] segment the raw data using the fixed sliding window method, and the classification model consists of concurrent

TABLE IX
COMPARISON BETWEEN THE STATE-OF-THE-ART HAR MODELS FOR
THE SBHARPT DATASET CONSIDERING THE TAS

Model	Accuracy	Segmentation Method	Limitations
Lone et al. [22]	93.05%	Fixed sliding window	The basic activity data is separated from transitional activity data and treated independently.
Chen et al. [5]	98.00%	Fixed sliding window	Treat all transitional activity classes as a single class.
Irfan et al. [24]	96.11%	Fixed sliding window	The experiments used a much smaller pre-segmented dataset which consists of statistical features matrix instead the raw data directly.
Yulita [38]	86.00%	Fixed sliding window	The experiments used a much smaller pre-segmented dataset which consists of statistical features matrix instead the raw data directly.
Noor et al. [27]	91.60%	Fixed sliding window	Limitations of fixed sliding window
Li et al. [6]	97.34%	Clustering-based	The proposed segmentation method is not applicable in real-time setting.
Noor et al. [25]	93.40%	Adaptive sliding window	The proposed segmentation method assumes the data is following Gaussian distribution.
Proposed approach	92.71%	Similarity-based	The difficulty of detecting the transitional activity which is in between two dynamic basic activities

feature learning pipelines to exploit the temporal relationships between the adjacent windows. However, the recognition accuracy is less than our proposed approach due to the limitation of the fixed sliding window. In summary, all the aforementioned works used the fixed sliding window for the segmentation process, which results in the suboptimal segmentation and the model's performance.

Improved segmentation methods have been proposed to overcome the limitations of the fixed sliding window. In [6], the raw data are segmented using the fixed sliding window method, and then, a cluster analysis method with K-means is used to classify the windows. The method is not applicable in real-time systems because it requires longitudinal sensor data to be analyzed and clustered to segment the signal, whereas the incoming sensor data need to be segmented immediately in real-time systems. In addition, data repetition causes the model to learn a different signal pattern than the actual pattern of the activity signal. Noor [25] proposed an adaptive sliding window method, whereby the window can be expanded based on the distribution of the data within the window. The proposed method assumes the distribution of the data to be Gaussian which may not be correct. Our proposed method does not make any assumptions about the data, whereby the segmentation completely relies on the pattern of the signals. In addition, the method requires computational resources to analyze the data and expand the window, which makes it unsuitable for real-time systems.

In summary, we find that the results of some studies have completely different settings compared to ours. These limitations make the comparison of the results with each other precisely unfair. Some studies have proposed improved segmentation methods; however, the methods are resource intensive and not applicable for real-time systems. The results obtained by our proposed approach are significant and valuable for advancing the field of HAR.

V. CONCLUSION

This article presents an SSA for the signal segmentation task. The proposed approach segments the sensor data and distinguishes between the BA windows and TA windows using the inner similarity and the temporal dissimilarity between adjacent windows. The proposed segmentation method feeds two CNN classifiers to perform the classification task. The first classifier aims to classify the BAs, and the other aims to classify the TAs.

For future work, the differentiation method of the temporal windows features proved to be effective in signal segmentation as well as the detection of TAs. Therefore, end-to-end models could be developed for both the segmentation and the recognition tasks. Finally, the proposed approach can be improved to enhance the detection of the TA that appears between two dynamic BAs. It is worth noting that, in principle, the proposed approach can be applied to dense sensing-based HAR, whereby the sensor data can be binary and/or multimodal data. This is because the nature of the data is time series, whereby segmentation is needed before the classification is performed. A suitable distance function can be specified based on the data type of the dataset.

REFERENCES

- [1] Z. Benhaili, Y. Balouki, and L. Moumoun, "A hybrid deep neural network for human activity recognition based on IoT sensors," *Int. J. Adv. Comput. Sci. Appl.*, vol. 12, no. 11, pp. 250–257, 2021, doi: [10.14569/IJACSA.2021.0121129](https://doi.org/10.14569/IJACSA.2021.0121129).
- [2] N. Alhammad and H. Al-Dossari, "Dynamic segmentation for physical activity recognition using a single wearable sensor," *Appl. Sci.*, vol. 11, no. 6, p. 2633, Mar. 2021, doi: [10.3390/app11062633](https://doi.org/10.3390/app11062633).
- [3] N. F. Ghazali, M. A. As'ari, N. Shahar, and H. F. M. Latip, "Investigation on the effect of different window size in segmentation for common sport activity," in *Proc. Int. Conf. Smart Comput. Electron. Enterprise (ICSCEE)*, Jul. 2018, pp. 1–7, doi: [10.1109/ICSCEE.2018.8538429](https://doi.org/10.1109/ICSCEE.2018.8538429).
- [4] U. Bermejo, A. Almeida, A. Bilbao-Jayo, and G. Azkune, "Embedding-based real-time change point detection with application to activity segmentation in smart home time series data," *Expert Syst. Appl.*, vol. 185, Dec. 2021, Art. no. 115641, doi: [10.1016/j.eswa.2021.115641](https://doi.org/10.1016/j.eswa.2021.115641).
- [5] L. Chen, S. Fan, V. Kumar, and Y. Jia, "A method of human activity recognition in transitional period," *Information*, vol. 11, no. 9, pp. 1–17, 2020, doi: [10.3390/info11090416](https://doi.org/10.3390/info11090416).
- [6] J.-H. Li, L. Tian, H. Wang, Y. An, K. Wang, and L. Yu, "Segmentation and recognition of basic and transitional activities for continuous physical human activity," *IEEE Access*, vol. 7, pp. 42565–42576, 2019, doi: [10.1109/ACCESS.2019.2905575](https://doi.org/10.1109/ACCESS.2019.2905575).
- [7] J.-L. Reyes-Ortiz, L. Oneto, A. Samà, X. Parra, and D. Anguita, "Transition-aware human activity recognition using smartphones," *Neurocomputing*, vol. 171, pp. 754–767, Jan. 2016, doi: [10.1016/j.neucom.2015.07.085](https://doi.org/10.1016/j.neucom.2015.07.085).
- [8] M. Wairagkar et al., "A novel approach for modelling and classifying sit-to-stand kinematics using inertial sensors," 2021, *arXiv:2107.06859*.
- [9] K. E. Pilario et al., "Wearables-based multi-task gait and activity segmentation using recurrent neural networks," *Sensors*, vol. 8, no. 1, pp. 1–11, 2020, doi: [10.3390/s20113117](https://doi.org/10.3390/s20113117).
- [10] P. Kumar and S. Chauhan, "Human activity recognition with deep learning: Overview, challenges and possibilities," *CCF Trans. Pervasive Comput. Interact.*, Jun. 2021, [Online]. Available: <https://www.preprints.org/doi/10.1007/s42486-021-00063-5>.
- [11] M. H. M. Noor, Z. Salicic, and K. I.-K. Wang, "Adaptive sliding window segmentation for physical activity recognition using a single tri-axial accelerometer," *Pervas. Mobile Comput.*, vol. 38, pp. 41–59, Jul. 2017, doi: [10.1016/j.pmcj.2016.09.009](https://doi.org/10.1016/j.pmcj.2016.09.009).
- [12] A. O. Jimale and M. H. M. Noor, "Subject variability in sensor-based activity recognition," *J. Ambient Intell. Humanized Comput.*, vol. 14, no. 4, pp. 3261–3274, 2021, doi: [10.1007/s12652-021-03465-6](https://doi.org/10.1007/s12652-021-03465-6).
- [13] S. Zhang et al., "Deep learning in human activity recognition with wearable sensors: A review on advances," *Sensors*, vol. 22, no. 4, p. 1476, Feb. 2022, doi: [10.3390/s22041476](https://doi.org/10.3390/s22041476).
- [14] B. A. Atalaa, A. Alenany, A. Helmi, and I. Ziedan, "Effect of data segmentation on the quality of human activity recognition," *East Afr. Scholars J. Eng. Comput. Sci.*, vol. 4480, no. 7, pp. 133–145, 2020.
- [15] J. Whitlock, O. Krand, and S. Jain, "Understanding activity segmentation for multi-sport competitions," in *Proc. 4th ACM Workshop Wearable Syst. Appl.*, Jun. 2018, pp. 16–20, doi: [10.1145/3211960.3211972](https://doi.org/10.1145/3211960.3211972).
- [16] C. F. Martindale, V. Christlein, P. Klumpp, and B. M. Eskofier, "Wearables-based multi-task gait and activity segmentation using recurrent neural networks," *Neurocomputing*, vol. 432, pp. 250–261, Apr. 2021, doi: [10.1016/j.neucom.2020.08.079](https://doi.org/10.1016/j.neucom.2020.08.079).
- [17] B. Fida, I. Bernabucci, D. Bibbo, S. Conforto, and M. Schmid, "Varying behavior of different window sizes on the classification of static and dynamic physical activities from a single accelerometer," *Med. Eng. Phys.*, vol. 37, no. 7, pp. 705–711, Jul. 2015, doi: [10.1016/j.medengphy.2015.04.005](https://doi.org/10.1016/j.medengphy.2015.04.005).
- [18] O. Banos, J.-M. Galvez, M. Damas, H. Pomares, and I. Rojas, "Window size impact in human activity recognition," *Sensors*, vol. 14, no. 4, pp. 6474–6499, Apr. 2014, doi: [10.3390/s140406474](https://doi.org/10.3390/s140406474).
- [19] A. Akbari, J. Wu, R. Grimsley, and R. Jafari, "Hierarchical signal segmentation and classification for accurate activity recognition," in *Proc. ACM Int. Joint Conf. Int. Symp. Pervasive Ubiquitous Comput. Wearable Comput.*, Oct. 2018, pp. 1596–1605, doi: [10.1145/3267305.3267528](https://doi.org/10.1145/3267305.3267528).
- [20] C. Xiao, Y. Lei, Y. Ma, F. Zhou, and Z. Qin, "DeepSeg: Deep-learning-based activity segmentation framework for activity recognition using WiFi," *IEEE Internet Things J.*, vol. 8, no. 7, pp. 5669–5681, Apr. 2021, doi: [10.1109/JIOT.2020.3033173](https://doi.org/10.1109/JIOT.2020.3033173).
- [21] J. He, Q. Zhang, L. Wang, and L. Pei, "Weakly supervised human activity recognition from wearable sensors by recurrent attention learning," *IEEE Sensors J.*, vol. 19, no. 6, pp. 2287–2297, Mar. 2019, doi: [10.1109/JSEN.2018.2885796](https://doi.org/10.1109/JSEN.2018.2885796).
- [22] K. J. Lone, L. Hussain, S. Saeed, A. Aslam, A. Maqbool, and F. M. Butt, "Detecting basic human activities and postural transition using robust machine learning techniques by applying dimensionality reduction methods," *Waves Random Complex Media*, vol. 2021, pp. 1–26, Sep. 2021, doi: [10.1080/17455030.2021.1971325](https://doi.org/10.1080/17455030.2021.1971325).
- [23] J. Shi, D. Zuo, and Z. Zhang, "Transition activity recognition system based on standard deviation trend analysis," *Sensors*, vol. 20, no. 11, pp. 1–11, 2020, doi: [10.3390/s20113117](https://doi.org/10.3390/s20113117).
- [24] S. Irfan, N. Anjum, N. Masood, A. S. Khattak, and N. Ramzan, "A novel hybrid deep learning model for human activity recognition based on transitional activities," *Sensors*, vol. 21, no. 24, pp. 1–20, 2021, doi: [10.3390/s21248227](https://doi.org/10.3390/s21248227).
- [25] M. H. M. Noor, "Feature learning using convolutional denoising autoencoder for activity recognition," *Neural Comput. Appl.*, vol. 2021, pp. 1–12, Jan. 2021, doi: [10.1007/s00521-020-05638-4](https://doi.org/10.1007/s00521-020-05638-4).
- [26] K. Xia, J. Huang, and H. Wang, "LSTM-CNN architecture for human activity recognition," *IEEE Access*, vol. 8, pp. 56855–56866, 2020, doi: [10.1109/ACCESS.2020.2982225](https://doi.org/10.1109/ACCESS.2020.2982225).
- [27] M. H. Mohd Noor, S. Y. Tan, and M. N. Ab Wahab, "Deep temporal Conv-LSTM for activity recognition," *Neural Process. Lett.*, vol. 54, no. 5, pp. 4027–4049, Oct. 2022, doi: [10.1007/s11063-022-10799-5](https://doi.org/10.1007/s11063-022-10799-5).
- [28] M. Rizwan and D. V. Anderson, (2016). *Comparison of Distance Metrics for Phoneme Classification Based On Deep Neural Network Features and Weighted K-NN Classifier*. [Online]. Available: <http://ttic.uchicago.edu/~klivescu/MLSLP2016/rizwan.pdf>
- [29] A. Ferrari, D. Micucci, M. Mobilio, and P. Napolitano, "Trends in human activity recognition using smartphones," *J. Reliable Intell. Environments*, vol. 7, no. 3, pp. 189–213, Sep. 2021, doi: [10.1007/s40860-021-00147-0](https://doi.org/10.1007/s40860-021-00147-0).
- [30] Y. Xu and T. T. Qiu, "Human activity recognition and embedded application based on convolutional neural network," *J. Artif. Intell. Technol.*, vol. 1, no. 1, pp. 51–60, Dec. 2020, doi: [10.37965/jait.2020.0051](https://doi.org/10.37965/jait.2020.0051).

- [31] K. Karagiannaki, A. Panousopoulou, and P. Tsakalides, "A benchmark study on feature selection for human activity recognition," in *Proc. ACM Int. Joint Conf. Pervasive Ubiquitous Computing: Adjunct*, Sep. 2016, pp. 105–108, doi: [10.1145/2968219.2971421](https://doi.org/10.1145/2968219.2971421).
- [32] D. Anguita et al., "A public domain dataset for human activity recognition using smartphones," in *Proc. Eur. Symp. Artif. Neural Netw.*, 2013.
- [33] I. Dirgová Luptáková, M. Kubovčík, and J. Pospíchal, "Wearable sensor-based human activity recognition with transformer model," *Sensors*, vol. 22, no. 5, p. 1911, Mar. 2022, doi: [10.3390/s22051911](https://doi.org/10.3390/s22051911).
- [34] M. Kljun, M. Teršek, and E. Štrumbelj, "A review and comparison of time series similarity measures," in *Proc. 29th Int. Electrotech. Comput. Sci. Conf.*, 2020, pp. 21–22.
- [35] K. Kavitha, B. Sandhya, and B. Thirumala, "Evaluation of distance measures for feature based image registration using AlexNet," *Int. J. Adv. Comput. Sci. Appl.*, vol. 9, no. 10, pp. 284–290, 2018, doi: [10.14569/IJACSA.2018.091034](https://doi.org/10.14569/IJACSA.2018.091034).
- [36] K. Labib, P. Uznanski, and D. Wolleb-Graf, "Hamming distance completeness," in *Proc. Annu. Symp. Combinat. Pattern Matching*, 2019, vol. 128, no. 14, pp. 1–14, doi: [10.4230/LIPIcs.CPM.2019.14](https://doi.org/10.4230/LIPIcs.CPM.2019.14).
- [37] S. S. Choi, S. H. Cha, and C. C. Tappert, "A survey of binary similarity and distance measures," in *Proc. 13th World Multi-Conf. Syst. Cybern. Inform.*, 2009, vol. 3, no. 1, pp. 80–85.
- [38] I. N. Yulita and S. Saori, "Human activities and postural transitions classification using support vector machine and k-nearest neighbor methods," *IOP Conf. Ser. Earth Environ. Sci.*, vol. 248, no. 1, 2019, Art. no. 012025, doi: [10.1088/1755-1315/248/1/012025](https://doi.org/10.1088/1755-1315/248/1/012025).



AbdulRahman M. A. Baraka received the B.S. degree in computer information system from Al-Quds Open University, Ramallah, Palestine, in 1999, and the M.S. degree in information technology from the Islamic University in Gaza, Gaza, Palestine, in 2011. He is currently pursuing the Ph.D. degree in computer science in Universiti Sains Malaysia, Minden, Malaysia.

He is currently a Lecturer with the Faculty of Technology and Applied Sciences, Al-Quds Open University. His research interests include deep learning and computer vision.



Mohd Halim Mohd Noor received the B.Eng. degree (Hons.) from International Islamic University Malaysia, Malaysia, in 2004, the M.Sc. degree from Universiti Sains Malaysia, George Town, Malaysia, in 2009, and the Ph.D. degree in computer systems engineering from the University of Auckland, Auckland, New Zealand, in 2017.

He is currently a Senior Lecturer with the School of Computer Sciences, Universiti Sains Malaysia, Minden, Malaysia. His main research interests include machine learning, deep learning, computer vision, and pervasive computing.